

Qiskit 양자 알고리즘 구현 문제집

게이트 단위 직접 구현으로 배우는 10단계 워크북
중학생도 따라갈 수 있도록 입력, 출력, 제한 조건, 채점 기준을 명확히 정리했습니다.

사용 방법

이 문제집의 목표는 Qiskit 문법을 외우는 것이 아니라, 알고리즘이 어떤 회로 구조로 만들어지는지 직접 확인하는 것입니다. 처음에는 완성된 알고리즘 클래스를 쓰지 말고 QuantumCircuit의 기본 게이트로 구현하세요. 각 문제는 작은 입력에서 시작해 상태벡터, 측정 분포, 회로 깊이를 함께 확인하도록 설계했습니다.

공통 입력 규칙

비트 문자열은 0과 1로만 작성합니다. 왼쪽 문자는 높은 번호 큐비트, 오른쪽 문자는 q_0 에 대응합니다. 예를 들어 101은 $q_2=1$, $q_1=0$, $q_0=1$ 을 뜻합니다. Qiskit 측정 문자열도 보통 이 순서로 읽습니다.

전체 로드맵

순서	알고리즘	핵심 개념	난이도
1	Deutsch 알고리즘	위상 킥백, 간섭	1 / 5
2	Deutsch-Jozsa 알고리즘	다중 큐비트 중첩, 오라클	2 / 5
3	Bernstein-Vazirani 알고리즘	숨은 비트 문자열	2 / 5
4	Grover 탐색	위상 오라클, 진폭 증폭	3 / 5
5	Simon 알고리즘	XOR 구조, mod 2 풀이	3 / 5
6	Quantum Fourier Transform	제어 위상, 비트 순서	3 / 5
7	Quantum Phase Estimation	controlled-U, inverse QFT	4 / 5
8	Quantum Counting	Grover + QPE	4 / 5
9	Shor 위수 찾기	모듈러 연산, 주기	5 / 5
10	VQE 또는 QAOA	양자-고전 최적화	4 / 5

공통 채점 습관

각 문제마다 Aer 시뮬레이터로 실행하고, 상태벡터 또는 측정 분포를 이론값과 비교하세요. 큐비트 수, 회로 깊이, 오라클 호출 횟수도 함께 출력하면 좋습니다.

1. Deutsch 알고리즘 | 난이도 1 / 5

문제 목표	1비트 함수가 constant인지 balanced인지 한 번의 오라클 호출로 판별한다.
입력 형식	oracle_type 가능한 값: constant_0, constant_1, identity, not
출력 형식	constant 또는 balanced
제한 조건	오라클 호출은 정확히 1회. 입력 레지스터 1큐비트, 보조 큐비트 1개를 사용한다.
예제 입출력	입력 identity 출력 balanced
구현 요구사항	네 가지 오라클 함수를 각각 만든다. 보조 큐비트는 $ 1\rangle$ 로 준비한 뒤 H를 적용한다. 마지막에는 입력 큐비트만 측정한다.
채점 기준	정확성 50점, 회로 구조 25점, 상태벡터 설명 15점, 코드 가독성 10점.

2. Deutsch-Jozsa 알고리즘 | 난이도 2 / 5

문제 목표	n비트 함수 $f(x)=a \cdot x \text{ xor } b$ 가 constant인지 balanced인지 판별한다.
입력 형식	n mask bias mask는 길이 n의 비트 문자열, bias는 0 또는 1.
출력 형식	constant 또는 balanced
제한 조건	$2 \leq n \leq 8$. mask가 000...0이면 constant, 아니면 balanced가 되어야 한다.
예제 입출력	입력 3 101 0 출력 balanced
구현 요구사항	mask의 1 위치마다 입력 큐비트에서 보조 큐비트로 CX를 건다. bias가 1이면 보조 큐비트에 X를 추가한다.
채점 기준	정확성 45점, 일반 n 지원 20점, 오라클 분리 20점, 출력 해석 15점.

3. Bernstein-Vazirani 알고리즘 | 난이도 2 / 5

문제 목표	숨겨진 비트 문자열 s 를 오라클 한 번으로 복구한다.
입력 형식	n $secret$ $secret$ 은 길이 n 의 비트 문자열.
출력 형식	$secret$ 과 같은 비트 문자열
제한 조건	$2 \leq n \leq 10$. $secret$ 은 0과 1로만 구성한다.
예제 입출력	입력 5 10110 출력 10110
구현 요구사항	$secret$ 의 1 위치마다 CX를 배치한다. 측정 문자열의 순서를 명확히 맞춘다.
채점 기준	정확성 50점, 비트 순서 처리 20점, 랜덤 $secret$ 테스트 20점, 코드 간결성 10점.

4. Grover 탐색 | 난이도 3 / 5

문제 목표	2^n 개의 상태 중 표시된 상태를 높은 확률로 찾는다.
입력 형식	n M $marked_state_1$... $marked_state_M$
출력 형식	측정 횟수 디셔너리. 가장 많이 나온 상태가 $marked_states$ 안에 있어야 한다.
제한 조건	$2 \leq n \leq 5$, $1 \leq M \leq 2^n$. GroverOperator 사용 금지.
예제 입출력	입력 3 1 101 출력 예 {'101': 962, '001': 14, '111': 12}
구현 요구사항	phase oracle과 diffuser를 직접 만든다. 반복 횟수는 $\text{round}(\pi/4 \cdot \sqrt{N/M})$ 근처로 정한다.
채점 기준	정답 확률 35점, 오라클 구현 25점, diffuser 구현 25점, 반복 횟수 계산 15점.

5. Simon 알고리즘 | 난이도 3 / 5

문제 목표	$f(x)=f(x \text{ xor } s)$ 를 만족하는 비밀 문자열 s 를 찾는다.
입력 형식	n $secret$ $secret$ 은 0이 아닌 길이 n 비트 문자열.
출력 형식	복구한 $secret$
제한 조건	$2 \leq n \leq 6$. 고전 후처리로 mod 2 가우스 소거를 구현한다.
예제 입출력	입력 4 1011 출력 1011
구현 요구사항	측정 결과 y 들이 $y \cdot s = 0 \pmod 2$ 를 만족하는지 확인한다. 충분한 독립 방정식을 모아 s 를 푼다.
채점 기준	오라클 25점, 측정 샘플 수집 20점, GF(2) 풀이 35점, 검증 코드 20점.

6. Quantum Fourier Transform | 난이도 3 / 5

문제 목표	계산 기저 상태에 QFT 또는 inverse QFT를 직접 적용한다.
입력 형식	n $basis_state$ $inverse$ $inverse$: 정방향 0, 역방향 1.
출력 형식	상태벡터 또는 확률 분포
제한 조건	$2 \leq n \leq 6$. qiskit.circuit.library.QFT 사용 금지.
예제 입출력	입력 3 101 0 출력 예 모든 3비트 상태가 확률 0.125
구현 요구사항	H, controlled phase, SWAP으로 직접 구성한다. 직접 구현과 Qiskit Statevector 결과를 비교한다.
채점 기준	게이트 순서 35점, inverse 지원 25점, 비트 순서 20점, 상태벡터 비교 20점.

7. Quantum Phase Estimation | 난이도 4 / 5

문제 목표	Phase 게이트 $U=P(2\pi\phi)$ 의 고유위상 ϕ 를 이진수로 추정한다.
입력 형식	precision phase_numerator $\phi = \text{phase_numerator} / 2^{\text{precision}}$
출력 형식	precision 길이의 비트 문자열
제한 조건	$2 \leq \text{precision} \leq 8, 0 \leq \text{phase_numerator} < 2^{\text{precision}}$.
예제 입출력	입력 3 3 출력 011
구현 요구사항	평가 큐비트에 H를 적용하고 $\text{controlled-}U^{(2^k)}$ 를 건 뒤 inverse QFT를 직접 적용한다.
채점 기준	controlled power 25점, inverse QFT 25점, 위상 해석 25점, 정확 테스트 25점.

8. Quantum Counting | 난이도 4 / 5

문제 목표	Grover operator의 위상을 추정해서 표시된 상태 개수 M을 추정한다.
입력 형식	n precision M marked_state_1 ... marked_state_M
출력 형식	추정한 정답 개수. 근삿값 허용.
제한 조건	$2 \leq n \leq 4, 3 \leq \text{precision} \leq 6$. Counting 전용 라이브러리 사용 금지.
예제 입출력	입력 3 4 2 001 110 출력 예 2.0
구현 요구사항	앞에서 만든 Grover oracle, diffuser, QPE를 결합한다. $M = N \sin^2(\theta)$ 를 이용해 개수를 계산한다.
채점 기준	구성 결합 35점, 수식 변환 25점, 여러 M 테스트 25점, 오차 설명 15점.

9. Shor 위수 찾기 | 난이도 5 / 5

문제 목표	$a^r = 1 \pmod N$ 을 만족하는 가장 작은 r 을 찾고, 가능하면 N 의 인수를 구한다.
입력 형식	N a precision
출력 형식	위수 r 또는 두 인수 $p\ q$
제한 조건	처음 목표는 $N=15, a=2$. $1 < a < N, \gcd(a,N)=1$.
예제 입출력	입력 15 2 8 출력 4 인수까지 출력하면 3 5
구현 요구사항	controlled modular multiplication, QPE, 연분수 복원, gcd 후처리를 분리해 구현한다.
채점 기준	위수 찾기 35점, 모듈러 회로 25점, 연분수 후처리 20점, 인수 검증 20점.

10. VQE 또는 QAOA | 난이도 4 / 5

문제 목표	하이브리드 양자-고전 최적화 알고리즘을 하나 구현한다.
입력 형식	VQE: n , Pauli 항 개수 K , 계수와 Pauli 문자열, layers QAOA: 정점 수 n , 간선 수 m , 레이어 p , 간선 목록
출력 형식	VQE는 최소 에너지와 파라미터. QAOA는 최적 비트 문자열과 cut 값.
제한 조건	VQE는 1~3큐비트, QAOA는 2~5정점부터 시작한다. scipy.optimize 사용 가능.
예제 입출력	VQE 입력 2 3 1.0 ZZ 1.0 XI 1.0 IX 2 QAOA 출력 예 0101 4.0
구현 요구사항	ansatz 또는 cost/mixer layer를 직접 만든다. 기대값 계산과 고전 최적화 루프를 분리한다.
채점 기준	회로 설계 30점, 기대값 계산 30점, 최적화 루프 25점, 결과 해석 15점.

최종 제출 체크리스트

1. 완성품 알고리즘 클래스 없이 직접 게이트를 배치했는가?
2. 입력 검증을 넣었는가? 예: 비트 문자열 길이, 0/1 문자, 범위 조건.
3. 측정 문자열의 순서를 설명했는가?
4. Aer 시뮬레이터 실행 결과를 저장했는가?
5. 상태벡터 또는 이론 분포와 비교했는가?
6. 오라클 호출 횟수와 회로 깊이를 출력했는가?
7. 실패하는 작은 반례를 최소 1개 이상 테스트했는가?

추천 진행 방식

먼저 함수 인자로 입력을 받아 구현하고, 알고리즘이 맞는 것을 확인한 뒤 `input()`이나 JSON 입력을 붙이세요. 회로 구현과 입력 파싱을 동시에 만들면 디버깅이 불필요하게 어려워집니다.